

Lineage-Aware Data Quality

Triaged, cost-bounded validation for multi-grain telemetry

Madiha Sahar • Evaluation & Reliability Case Study • runnable reference implementation, synthetic data

The problem. Data quality usually fails two ways, both expensive. The *scream test* — ship it and wait for a downstream user to complain — costs nothing until it costs everything. The brute-force fix — check every stream, every 15 minutes, at every statistic — runs to **~\$68k/month** and bottlenecks ingestion. This system lives in between.

THE IDEA

Data quality is not free, so it must be **triaged**. A stream's importance is inherited, not intrinsic: it matters because of what depends on it. Walk the **lineage**, compute each stream's **blast radius** (criticality-weighted downstream reach), and spend QA compute only where the blast radius justifies it.

RESULT

- | **85% less QA spend:** \$9,960/mo vs \$68,000/mo brute-force baseline
- | **2 silent faults** caught that aggregate monitoring reported as GREEN
- | **Deterministic verdict** is a count of falsifiable checks, not a model's opinion

WHAT AGGREGATES HIDE

- **Segment-level MNAR gap:** one segment goes dark while global volume still reads 84% healthy. Only a *conditional, per-segment* check finds it.
- **Tail drift:** p99 degrades while mean & p50 stay flat. Only a *targeted percentile* check finds it — and percentiles are expensive, so they run only where a tail matters downstream.

THE ENGINE — VALUE VS COST, PER CHECK

value = blast_radius × prior_fault_rate × detection_severity | cost = check_units × volume × runs_per_month(cadence)

Keep the cheapest cadence whose value clears its cost; drop otherwise. The expensive tail check is scheduled **only** on streams feeding a tail-sensitive consumer.

ACT 3 – NAIVE AGGREGATE CHECKS (cheap, and they lie)

```
session_events_log OK volume 84.1% of expected <- GREEN, but 'west' is dark
>> silently missed: edge_latency p99 tail drift + session_events MNAR gap
```

ACT 5 – TARGETED CHECKS RUN (catch what aggregates hid) + VERDICT

```
[CAUGHT] edge_latency 15m tail_percentiles p99=188ms (ceiling 130) | mean=46 p50=41 look fine
[CAUGHT] session_events_log completeness cond segment(s) dark: west
revenue_dashboard trust=73% DEGRADED, clear-by ~90m
exec_weekly_report trust=100% TRUSTED, all protecting checks held
```

THE VERDICT LAYER

Output is a **trust ratio** per consumer — severity-weighted pass rate over the checks protecting the streams it reads — with blast radius and an estimated clear-by time. Not "something moved," but "how much to trust this, which checks failed, and until when." Auditable by construction.

[Lineage / blast radius](#)[Conditional MNAR](#)[Tail percentiles](#)[Cost-bounded triage](#)[Deterministic verdict](#)

Runnable on synthetic data — plants the faults and catches them in one command. Generalizes production work replacing standard-deviation alerting with lineage-driven data contract validation.